

MASTER OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Mitigating Free-Riding Behavior in Federated Learning Systems

Aashnan Rahman

231041006

Department of Computer Science and Engineering

Islamic University of Technology

July, 2026

Mitigating Free-Riding Behavior in Federated Learning Systems

Aashnan Rahman

231041006

Department of Computer Science and Engineering

Islamic University of Technology

July, 2026

Certificate of Approval

The thesis titled, *Mitigating Free-Riding Behavior in Federated Learning Systems* submitted by Aashnan Rahman with student ID 231041006 of Academic Year 2024–25 has been found as satisfactory and accepted as partial fulfillment of the requirement for the degree Master of Science in Computer Science and Engineering on July 04, 2026.

Board of Examiners:

Dr. Md. Azam Hossain

Associate Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Chairman
(Supervisor)

Dr. Mike Hunt

Professor

Department of Electrical and Computer Engineering

Imaginary University of Technology, Texas, US

Member
(Co-Supervisor)

Dr. Justin Case

Professor and Head and Head

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member
(Ex-Officio)

Dr. Benjamin Dover

Assistant Professor and Head

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member

Member

Mr. Noah Pinion

Assistant Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member
(External)

Toby Lerone

Professor and Head

Department of Computer Science and Engineering

Bjork University, UK

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by Aashnan Rahman under the supervision of Dr. Md. Azam Hossain, Associate Professor, Department of Computer Science and Engineering, Islamic University of Technology (IUT), Dhaka, Bangladesh and co-supervision of Dr. Mike Hunt, Professor, Department of Electrical and Computer Engineering, Imaginary University of Technology, Texas, US . It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Dr. Md. Azam Hossain

Associate Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur,

Bangladesh

Date: July 04, 2026

Aashnan Rahman

Student ID: 231041006

Date: July 04, 2026

Dr. Mike Hunt

Professor

Department of Electrical and Computer Engineering

Imaginary University of Technology, Texas, US

Date: July 04, 2026

Dedicated to To be added later

Contents

1	Introduction	1
1.1	Introduction to Federated Learning and the Free Rider Problem	1
1.2	Scope and Motivation	2
1.3	Problem Statement	3
1.4	Research Challenges	4
1.5	Contributions	6
1.5.1	Organization	7
2	Related Work	8
2.1	Background	8
2.2	Foundational Characterization of Free-Rider Attacks	8
2.3	Advanced and Stealthy Free-Rider Attacks	9
2.4	Server-Side Detection and Audit-Based Mitigation	10
2.5	Privacy-Attack-Based Detection	11
2.6	Hardware-Based and Cryptographic Solutions	13
2.7	Other Notable Approaches	13
2.8	Summary and Research Gap	14
3	Proposed Methodology	16
3.1	Design Objective and Information Boundary	16
3.2	System and Threat Model	17
3.3	Robust Two-Signal Detector	17
3.3.1	Two-sided magnitude evidence	17
3.3.2	Scale-free layer profile	18
3.3.3	Exact send-back evidence	18
3.4	Client States and Evidence Accounting	18
3.5	Probing Schedule	19
3.5.1	Warm-up	19

3.5.2	Initial double coverage sweep	19
3.5.3	Post-coverage surveillance	20
3.5.4	Candidate and suspicion probing	20
3.6	Rotating Anchor Roster	21
3.7	Aggregation, Removal, and Checkpointing	21
3.8	Computational Characteristics and Limitations	21
3.9	Summary	22
4	Results and Discussion	23
4.1	Experimental Questions	23
4.2	Experimental Setup	23
4.3	IID Results	24
4.4	Non-IID Results	24
4.5	Anchor Contamination Case Study	25
4.6	False Positives and Seed-Specific Clients	25
4.7	Model Utility	26
4.8	Runtime and Resource Use	26
4.9	Threats to Validity	27
4.10	Overall Interpretation	27
5	Conclusion	28
	References	30

List of Figures

List of Tables

2.1	Comparative summary of existing free-rider mitigation approaches . .	14
3.1	Principal SWT-CP v8 parameters	19
4.1	Aggregate v8 results across FR1–FR4 for each setting	24
4.2	Detailed non-IID results (free riders removed / honest clients removed)	24
4.3	Execution profile aggregated across four attacks	26

List of Abbreviations

FL	Federated Learning
FR	Free Rider
IID	Independent and Identically Distributed
MAD	Median Absolute Deviation
SWT-CP	Suspicion-Weighted Trap Detection with Cumulative Penalties
FedAvg	Federated Averaging

Acknowledgement

I am profoundly grateful to my supervisor, Dr. Lou Natic, for his endless patience, insightful critiques, and for always knowing just when to say, “Have you tried turning it off and on again?” His expertise and encouragement were instrumental in the completion of this thesis.

A special thanks to my co-supervisor, Dr. Sue Permann, who provided invaluable support and the occasional pep talk disguised as a lecture on time management. Her unwavering belief in my work kept me going, even when the only thing I believed in was the power of procrastination.

Finally, to my parents, whose love and support have been a constant source of strength. Thank you for always being there, for pretending to understand my research when I rambled on, and for not asking too many questions about why I was still not employed after all these years.

To all of you, I extend my heartfelt appreciation and a promise: next time, I will take fewer detours through the land of streaming services.

Abstract

Federated learning allows clients to benefit from a shared model without centralizing raw data, but the server cannot directly verify whether a submitted update resulted from genuine local training. This thesis presents Suspicion-Weighted Trap Detection with Cumulative Penalties (SWT-CP), a lightweight server-side mechanism that secretly replaces the global model with a randomized trap model for selected clients and evaluates the response using robust update-norm and layer-profile statistics. SWT-CP combines a two-trap coverage sweep, candidate confirmation, frequent re-probing of suspicious clients, rotating anchor panels, continued surveillance, cumulative penalties, and evidence-based rehabilitation. Detection uses only returned model parameters; client-reported accuracy, loss, training time, and resource use are not trusted inputs.

SWT-CP v8 was evaluated on MNIST with 100 clients and 100 communication rounds under IID and Dirichlet non-IID partitions ($\alpha = 0.5$), four free-rider strategies, attacker shares of 30% and 40%, and seeds 42 and 43. Across all IID configurations, all 560 free riders were removed, with 15 honest removals; IID with 40% attackers and seed 43 achieved perfect removal precision and recall. Under non-IID data, FR1–FR3 retained 100% recall, but false removals increased and adaptive FR4 exposed a contamination limit. FR4 was fully detected at 30% attackers for both seeds, yet no FR4 client was removed at 40% for either seed. Undetected attackers entered the nominally clean anchor roster and, in one observed case, formed malicious-majority anchor panels that contributed directly to an honest removal. Final global accuracy remained between 98.82% and 98.95%.

The findings show that secret statistical probing is effective and inexpensive for replay, random-update, and historical-average attacks, but that an unflagged client is not necessarily a trustworthy reference and robust statistics alone do not guarantee resistance to concentrated adaptive attackers under heterogeneous data. SWT-CP is therefore reported as a strong lightweight baseline with a clearly identified non-IID, high-contamination failure mode rather than as a universal proof of client computation.

Chapter 1

Introduction

1.1 Introduction to Federated Learning and the Free Rider Problem

Federated Learning (FL) has emerged as a transformative paradigm for distributed machine learning that enables multiple clients such as mobile devices, edge servers, or organizational silos to collaboratively train a shared global model without exposing their raw, locally stored data [7], [9]. Unlike traditional centralized machine learning, where data must be aggregated on a single server, FL operates on a fundamentally different principle: *the model travels to the data, not the other way around*.

In a typical FL architecture, a central server initializes a global model and distributes it to a set of participating clients. Each client independently trains the model on its local dataset for a fixed number of epochs and computes a local model update (e.g., gradients or updated weights). These updates are transmitted back to the server, which aggregates them—commonly via the Federated Averaging (FedAvg) algorithm—to produce an improved global model [9]. This process is repeated over multiple communication rounds until the global model converges to a satisfactory performance level.

This architecture offers several compelling advantages:

- **Privacy Preservation:** Since raw data never leaves the client device, FL inherently reduces the risk of data leakage and aligns with data protection regulations such as GDPR and HIPAA.
- **Reduced Communication Overhead:** Only model parameters or gradients

are exchanged, rather than entire datasets, which is particularly advantageous in bandwidth-constrained environments.

- **Decentralized Data Utilization:** FL enables organizations and individuals to benefit from collaborative model training without relinquishing control over sensitive data.
- **Scalability:** FL systems can, in principle, incorporate thousands to millions of participating clients, as seen in large-scale mobile deployments such as Google’s keyboard prediction systems [2], [5].

Privacy is not merely a desirable feature of FL it is often the primary motivation for its adoption. In domains such as healthcare, finance, and personal mobile applications, the ability to train robust models while keeping sensitive data on-device is a critical enabler of machine learning adoption in regulated and privacy-sensitive contexts.

However, the very decentralization and privacy guarantees that make FL attractive also introduce a unique vulnerability: the server has limited visibility into what each client is actually doing locally. This opacity creates an opportunity for malicious or self-interested clients to exploit the system through what is known as **Free Rider (FR) behavior**. A free rider is a participant that contributes little or no genuine computational effort toward local model training, yet still submits an update to the server in order to receive the benefits of the collaboratively trained global model—effectively benefiting from the effort of honest participants while incurring minimal cost. Because free riders can fabricate or manipulate updates that superficially resemble legitimate contributions, they are often difficult to distinguish from genuine, well-behaved clients using standard aggregation mechanisms alone.

1.2 Scope and Motivation

To understand why free rider behavior poses a serious threat to FL systems, consider a simple real-world analogy. Suppose a group project involves ten students who are collectively assigned a task, and their final grade depends on the group’s combined output. If six students genuinely contribute their effort while the remaining four merely submit superficial or copied work without doing any real analysis, the final result may still appear complete and acceptable to an external evaluator who only inspects the submitted output. Yet the four free-riding students receive the same reward (the grade) as the diligent six, despite having contributed little of value. Over time, this dynamic discourages honest participants, degrades the overall quality of the group’s

output, and undermines the fairness of the collaborative process.

This scenario maps directly onto FL systems. Free-riding clients can submit updates that are statistically or structurally similar to genuine updates—for instance, by adding small perturbations to the previous global model, reusing outdated updates, or applying minimal or synthetic training—without performing meaningful local training on their own data. Because the FL server typically aggregates updates based on volume or simple heuristics (e.g., number of local samples reported by the client) rather than verifying the training process itself, such free riders can successfully "blend in" with genuine contributors. They benefit from the improved global model at every round while incurring negligible computational, energy, or data costs.

The consequences of unmitigated free-rider behavior in FL systems are significant:

- **Degraded Model Quality:** A high proportion of free riders can slow convergence or degrade the accuracy of the global model, especially in non-IID data settings where genuine diverse contributions are essential.
- **Unfair Resource Allocation:** In incentive-based FL systems, free riders may unfairly claim rewards (monetary or otherwise) intended to compensate genuine contributors, discouraging honest participation over time.
- **Security and Trust Erosion:** Persistent free riding undermines trust in the FL ecosystem, potentially disincentivizing high-quality clients from participating altogether.
- **Scalability of Malicious Behavior:** As FL systems scale to include thousands of clients, manual or centralized verification of each client's genuine contribution becomes computationally and practically infeasible, making the free rider problem increasingly urgent.

Given the growing deployment of FL in real-world, large-scale, and often adversarial environments, addressing free rider behavior is not merely an academic exercise but a practical necessity for ensuring the long-term viability, fairness, and robustness of federated systems.

1.3 Problem Statement

The core problem addressed in this thesis can be formally framed as follows:

In a Federated Learning system involving N clients participating over T communication rounds, a subset of clients $\mathcal{F} \subset \mathcal{N}$ may behave as free rid-

ers—submitting model updates that do not reflect genuine training on locally available data, while still appearing statistically and structurally consistent with updates submitted by honest clients. The central server, having no direct access to client-side data or training logs, must identify or mitigate the influence of these free-riding clients using only the information available through the exchanged model updates, aggregation metadata, and system-level observations, without compromising the privacy guarantees that FL is designed to provide.

This problem is inherently challenging because it sits at the intersection of two competing goals: (1) verifying the authenticity and quality of client contributions, and (2) preserving the privacy-preserving nature of the FL paradigm, which explicitly avoids direct inspection of client data or computation. A free rider detection or mitigation mechanism that requires access to raw client data, imposes excessive computational burden, or fails to generalize across dynamic and heterogeneous client populations is, in practice, unsuitable for real-world FL deployments.

1.4 Research Challenges

Detecting and mitigating free rider behavior in FL systems presents several interrelated challenges:

1. **Indistinguishability of Updates:** Free riders can craft updates through techniques such as noise injection, weight perturbation, or reuse of previous global models—that closely mimic the statistical properties of genuine updates, making detection based on superficial inspection unreliable.
2. **Absence of Ground Truth:** The server cannot directly observe a client’s local training process, dataset, or computational effort, making it difficult to establish a reliable baseline against which suspicious updates can be compared.
3. **Non-IID Data Distributions:** In realistic FL settings, client data is often non-independent and non-identically distributed (non-IID), a well-known source of optimization instability and client-update diversity in FL [7], [15]. Legitimate updates from clients with unusual or minority data distributions may appear anomalous, increasing the risk of falsely flagging honest clients as free riders.
4. **Privacy Constraints:** Any detection mechanism must operate without requiring access to raw client data or intrusive monitoring, as doing so would violate the fundamental privacy guarantees that motivate the use of FL in the first place.

This constraint becomes even stronger in deployments that rely on secure aggregation, where the server is deliberately prevented from inspecting individual client updates [1].

5. **Scalability:** FL systems may involve thousands to millions of clients. Detection mechanisms that rely on computationally expensive verification, pairwise comparisons, or centralized auditing do not scale efficiently to such large populations.
6. **Dynamic and Adaptive Adversaries:** Free riders may adapt their strategies over time to evade detection (e.g., alternating between genuine and free-riding behavior), requiring mitigation techniques that are robust to adaptive and evolving attack patterns rather than static, one-time detection.
7. **Trade-off Between Robustness and Model Utility:** Overly aggressive filtering or penalization of suspicious clients risks excluding legitimate contributions, particularly from clients with limited resources or atypical data, thereby harming overall model utility and fairness.

Several existing approaches have attempted to address aspects of this problem, but each carries notable limitations:

- **Hardware-based methods**, such as trusted execution environments (TEEs) or secure hardware attestation, can verify that computation genuinely occurred on a client device. However, these methods require specialized hardware support, are costly to deploy at scale, and are often infeasible for heterogeneous, resource-constrained edge devices.
- **Proof-of-work approaches**, inspired by blockchain consensus mechanisms, require clients to demonstrate computational effort before their updates are accepted. While effective in principle, these methods impose significant computational and energy overhead, which is particularly problematic for battery-constrained mobile or IoT devices, and does not directly guarantee that the effort corresponds to genuine, useful model training.
- **Server-side verification** techniques attempt to validate updates by re-executing or partially replicating client-side computation. These approaches, however, often require access to client data or proxies thereof, directly conflicting with FL's privacy-preserving objectives and with secure-aggregation-style deployments [1], and introduce substantial computational burden on the server.
- **Computation checking** methods analyze metadata such as training time, num-

ber of local epochs, or resource usage logs reported by clients. Such self-reported metrics are inherently susceptible to falsification by malicious clients, limiting their reliability as a standalone defense.

Collectively, these limitations—high cost, privacy compromise, poor scalability, and susceptibility to adaptive adversaries—highlight the need for a fundamentally different approach: one that is lightweight, privacy-preserving, scalable to large and dynamic client populations, and robust against evolving free-riding strategies.

1.5 Contributions

Motivated by the limitations of existing free rider mitigation strategies, this thesis proposes a lightweight, privacy-preserving, and scalable framework for detecting and mitigating free rider behavior in Federated Learning systems. The key contributions of this work are summarized as follows:

1. **A Privacy-Preserving Detection Mechanism:** We design a detection approach that relies solely on information already available at the server through the standard FL communication protocol—such as update characteristics and aggregation-level statistics—without requiring access to raw client data, specialized hardware, or intrusive auditing procedures.
2. **Lightweight Computational Overhead:** Unlike proof-of-work or server-side re-execution methods, the proposed approach introduces minimal additional computational and communication cost, making it practical for deployment in resource-constrained and large-scale FL environments.
3. **Robustness to Dynamic and Adaptive Free Riders:** The proposed method is designed to adapt to evolving free-riding strategies over multiple communication rounds, rather than relying on static, one-time verification, thereby improving robustness against adaptive adversarial behavior.
4. **Preservation of Model Utility for Honest Clients:** By carefully balancing detection sensitivity against the risk of false positives, particularly in non-IID settings, the proposed approach aims to mitigate free rider influence without unfairly penalizing legitimate clients with atypical but genuine data distributions.
5. **Scalability to Large Client Populations:** The proposed framework is designed with computational and communication efficiency in mind, enabling its appli-

cation to FL systems involving large and dynamically changing sets of participating clients.

6. **Empirical Validation and Failure Analysis:** We evaluate four free-rider strategies on MNIST under IID and Dirichlet non-IID partitions, two attacker shares, and two random seeds. Beyond precision, recall, F1, model accuracy, and runtime, the analysis traces false removals to individual clients and identifies a reproducible FR4 failure caused by statistical and anchor-set contamination at 40% non-IID attackers. No unimplemented comparison with external baselines is claimed.

Collectively, these contributions aim to advance the practical deployability of Federated Learning systems by addressing one of its most persistent and underexplored vulnerabilities, offering a solution that preserves the core privacy and efficiency benefits that make FL attractive in the first place.

1.5.1 Organization

Chapter 1 introduces the free-rider problem, research challenges, and contributions. Chapter 2 reviews statistical, learned, audit-based, privacy-attack-based, and hardware-assisted defenses. Chapter 3 specifies SWT-CP v8, including its threat model, two-signal detector, probing schedule, client states, penalties, rehabilitation rule, and aggregation policy. Chapter 4 presents the experimental protocol and analyzes IID and non-IID results, model utility, runtime, false positives, and the FR4 contamination failure. Chapter 5 summarizes the findings, limitations, and priorities for future work.

Chapter 2

Related Work

2.1 Background

Research on free-rider (FR) behavior in Federated Learning (FL) can be organized into five broad categories: (1) foundational work that formalizes free-rider attacks and analyzes their effect on convergence, (2) increasingly stealthy attack constructions that exploit the geometric and statistical properties of honest updates, (3) server-side detection and audit-based mitigation mechanisms, (4) detection methods that repurpose privacy-attack techniques, and (5) hardware-based and cryptographic verification schemes. This section reviews each category in turn, compares the underlying assumptions and effectiveness of representative methods, and concludes by identifying the gap that motivates the approach proposed in this thesis.

2.2 Foundational Characterization of Free-Rider Attacks

Fraboni et al. [3] provided the first theoretical and experimental analysis of free-rider attacks against iterative parameter-aggregation schemes such as FedAvg and FedProx. Modeling stochastic gradient descent as a continuous-time process, the authors formally proved that a free rider’s fabricated update can be constructed to provably converge toward the aggregated model produced by fair participants, meaning the attacker obtains a fully-trained global model without contributing any data. They first showed that the most naive attack—simply returning the previous global model unchanged—is trivially detectable by basic server-side checks (e.g., zero update norm). To evade such checks, they proposed a *disguised free-rider* strategy that adds time-

decaying Gaussian noise to the previous global model, with the noise variance calibrated to the empirical standard deviation of the global gradient so that the fabricated update statistically mimics the natural shrinkage of SGD updates as training converges. Their experiments across both IID and non-IID settings showed that this disguised attack is substantially harder to detect than the naive variant, and they concluded with practical recommendations for FL deployments in sensitive, high-value domains such as healthcare.

Complementing this, Lin et al. [8] were the first to formally introduce the notion of free-rider attacks in FL and to jointly study attack construction and detection. They proposed the *delta weights* attack, in which a free rider constructs its fake update by adding calibrated noise to the previous round’s global gradient rather than to the model weights directly, producing updates that more closely resemble genuine gradient-based contributions than naive random-weight submissions. On the defense side, they proposed **STD-DAGMM**, a detection method that augments a Deep Autoencoding Gaussian Mixture Model (DAGMM) with the standard deviation of each client’s update as an additional feature, framing free-rider detection as a high-dimensional anomaly-detection problem. They further extended their analysis to scenarios involving multiple simultaneous free riders and the presence of differential privacy noise, showing that the interaction between privacy-preserving noise and free-rider disguise strategies further complicates detection—an open problem they explicitly flagged for future research.

Together, these two works established that (i) free-riding is a concretely realizable and theoretically grounded threat rather than a hypothetical one, and (ii) naive server-side aggregation, and even simple statistical checks, are insufficient once an attacker calibrates its fabricated update to mimic the stochastic behavior of genuine local training.

2.3 Advanced and Stealthy Free-Rider Attacks

Building directly on the disguising strategies above, Zhu et al. [16] introduced a more sophisticated *advanced free-rider attack* that explicitly targets the geometric properties exploited by feature-based defenses. The authors analyzed the expected cosine similarity and ℓ_2 -norm relationships between honest client updates and the converging global model in IID settings, deriving closed-form approximations for how these quantities evolve as training progresses. Using these derived relationships, their attack injects calibrated Gaussian noise into only a *subset* of the global gradient’s parameters, with the noise magnitude explicitly scaled so that the resulting fake update

reproduces the cosine-similarity and norm statistics expected of a genuine client update at that point in training. Because this attack is constructed to match the exact geometric signature that earlier detection mechanisms (including cosine-similarity and ℓ_2 /standard-deviation-based checks) rely on, it was shown—both in the original work and in later independent evaluations [11]—to substantially degrade the effectiveness of feature-based detectors, particularly in IID scenarios where honest client updates are naturally similar to one another and thus offer the attacker less statistical cover to hide *behind* but also less distinctiveness to be caught *by*.

This line of work demonstrates a critical limitation of static, feature-based defenses: any detection rule based on a fixed set of update statistics (norm, standard deviation, cosine similarity) can, in principle, be reverse-engineered and matched by an adaptive attacker with knowledge of the aggregation dynamics. It motivates the need for detection mechanisms that target properties of genuine training which are fundamentally difficult to fabricate, rather than statistical proxies that a sufficiently informed adversary can approximate.

2.4 Server-Side Detection and Audit-Based Mitigation

A second line of work designs server-side detection pipelines that combine anomaly detection with auxiliary contribution or reputation signals. Wang et al. proposed **FRAD** [13], a free-rider attack detection mechanism targeted at FL deployments in the Artificial Intelligence of Things (AIoT), where device heterogeneity is pronounced. FRAD combines a DAGMM-based anomaly detector (extending the STD-DAGMM approach of Lin et al.) with an explicit *contribution model* that estimates each device’s expected contribution from its computing resources, communication cost, and data quality, and a *reputation model* based on the PageRank algorithm that iteratively re-weights clients according to their historical behavior. By fusing statistical anomaly detection with resource- and reputation-aware contribution modeling, FRAD is explicitly designed to select benign nodes for participation even under conditions of information asymmetry between the server and heterogeneous IoT clients. However, because its detection backbone remains a DAGMM-style autoencoder, it inherits the same scalability limitation identified in later work [11]: the memory and computational cost of training an autoencoder over every client’s full (flattened) update grows with both model size and the number of clients, which becomes prohibitive for larger neural networks or large client populations.

Wang et al. also proposed **PASS** [14], a Parameter Audit-based Secure and fair FL

Scheme, motivated by the observation that prior defenses generally assumed adversaries constitute a minority of clients and focused narrowly on *anonymous* free riders (clients lacking data or compute) while losing effectiveness against *selfish* free riders (clients that possess adequate resources but strategically withhold them). PASS combines a privacy-preserving auditing strategy—injecting weak differential-privacy noise via a Gaussian mechanism together with a parameter-pruning step to protect client data during the audit—with a novel contribution-evaluation method used to assess each client’s genuine performance. The scheme’s experimental evaluation demonstrated that PASS achieves privacy protection comparable to state-of-the-art baselines with limited accuracy loss, attains a higher defense success rate, lower false-positive rate, and higher F1-score than prior methods against both anonymous and selfish free-rider attacks, and—notably—remains effective even when adversarial clients constitute a *majority* of the population (F1-scores of 89% and 87% against anonymous and selfish attacks, respectively, even when adversaries exceed 50% of clients). This robustness to majority-adversary settings distinguishes PASS from earlier defenses, though its parameter-auditing procedure introduces non-trivial computational overhead at the server compared to lightweight statistical baselines.

Collectively, FRAD and PASS demonstrate that meaningful detection is achievable using only server-observable signals augmented with lightweight contribution modeling, aligning with FL’s privacy-preserving ethos. However, both rely on carefully engineered statistical or audit pipelines whose effectiveness depends on assumptions (e.g., DAGMM’s density-estimation quality, or the calibration of PASS’s audit noise) that may require retuning as data distributions, model architectures, and free-rider strategies evolve.

2.5 Privacy-Attack-Based Detection

A more recent line of work, exemplified by Recasens et al.’s **FRIDA** (Free-Rider detection using privacy Attacks) [11], departs from indirect statistical proxies and instead repurposes *privacy-attack* methodologies to directly verify whether a client’s update reflects genuine training. The key insight is that prior detection families—feature-based statistics (norm, standard deviation, cosine similarity), DAGMM-based anomaly detection (including FRAD), log-based auditing, and incentive/reputation mechanisms—all rely on *indirect effects* of free-riding rather than on the root cause: the absence of genuine training on real client data. FRIDA instead proposes four detection mechanisms built on two families of privacy attack. The first family adapts *membership inference attacks* (MIA), originally studied as a way to infer whether a record was part of a model’s

training data [12]: the server distributes a small shared “canary” dataset that honest clients are required to additionally train on each round; a *loss-based* detector then flags clients whose canary-set loss fails to decrease as expected, while a *cosine-based* detector compares the cosine similarity of a client’s gradient with canary-sample gradients against non-canary samples via a statistical test, flagging clients for whom the two distributions are indistinguishable (indicating no canary training occurred). The second family adapts *property inference attacks* (PIA), which exploit information leakage from collaborative model updates to infer aggregate properties of a participant’s local data [10], to estimate each client’s local label distribution from its submitted gradients without requiring any shared dataset or additional client-side training; a *consistency-based* detector flags clients whose inferred label distribution changes anomalously across rounds, while a *diversity-based* detector flags clients whose inferred distribution is best explained as a combination of pure noise and the global label distribution—the signature of a fabricated, rather than genuinely trained, update.

The authors’ extensive evaluation across CIFAR-10 and CIFAR-100 with AlexNet and VGG-19 architectures, under both IID and non-IID data (Dirichlet-distributed with $\alpha \in \{0.1, 1.0\}$), and against all three attack strategies discussed above (Fraboni et al., Lin et al., and Zhu et al.), showed that FRIDA matches or outperforms feature-based and DAGMM-based baselines in nearly every setting, with especially pronounced advantages in IID scenarios and on more complex datasets—precisely the regimes in which the advanced attack of Zhu et al. is most effective at evading feature-based detectors. Notably, PIA-based detection was shown to remain robust even when honest clients apply local differential privacy to their updates, since it exploits dataset-level statistical properties rather than fine-grained individual-level signals, whereas MIA-based detection degrades sharply under such privacy noise. However, FRIDA’s design assumes the server has visibility into individual client updates, which is fundamentally in tension with secure-aggregation-based privacy protection; the authors propose a two-server architecture as a partial mitigation but acknowledge this adds system complexity, and the cosine-based MIA variant was shown to be evadable by an adaptive, resource-endowed free rider willing to train specifically on the canary set. The MIA-based variants also introduce non-trivial per-round computational cost relative to lightweight feature-based statistics, and PIA-based detection shifts additional computational burden onto the server.

2.6 Hardware-Based and Cryptographic Solutions

In parallel with statistical and inference-based defenses, several works have explored hardware-based and cryptographic approaches. Trusted Execution Environments (TEEs), such as Intel SGX, have been proposed to provide attestable, tamper-evident proof that local training genuinely executed on a client device. Similarly, blockchain-based FL frameworks have incorporated proof-of-work or proof-of-training consensus mechanisms that require clients to demonstrate verifiable computational effort before their updates are accepted into aggregation, and secure-aggregation or zero-knowledge-proof-based schemes have been proposed to protect or verify client-side computation without exposing raw client data or full individual updates [1].

While such approaches can offer strong, cryptographically verifiable guarantees against fabricated updates, they carry substantial practical costs. TEE-based solutions require specialized hardware support that is unavailable across much of the heterogeneous device landscape typical of real-world FL (older mobile devices, low-cost IoT sensors), limiting deployability. Proof-of-work mechanisms impose significant computational and energy overhead that is particularly problematic for battery-constrained edge devices and works directly against FL’s goal of enabling participation from resource-limited clients. Zero-knowledge-proof schemes, while avoiding the hardware-dependency problem, typically introduce substantial proof-generation and verification overhead that scales poorly with model size. None of these approaches inherently addresses the non-IID, dynamic nature of real-world client populations, and all introduce deployment complexity that has, to date, limited their adoption outside specialized or high-value settings.

2.7 Other Notable Approaches

Beyond the categories above, several related lines of work address free riding indirectly through contribution evaluation and incentive design. As noted by Recasens et al. [11], free-rider detection can be framed as an extreme case of the broader contribution-evaluation problem, for which the Shapley value is the game-theoretic gold standard; however, exact computation is combinatorially intractable, and approximations such as Data Shapley and task-specific Shapley valuation are generally designed to *rank* contributors rather than precisely identify zero-contribution participants [4], [6], limiting their direct applicability to free-rider detection. Log-based auditing schemes detect free riders by inspecting client activity or stake logs rather than model updates directly, while incentive- and reputation-based frameworks allocate rewards or

model access proportionally to accumulated trust across rounds. These approaches are complementary to update-based detection but operate on information beyond the raw model updates (e.g., detailed activity logs or reward accounting) and remain vulnerable to strategic clients who behave honestly during an initial trust-building phase before defecting once sufficient reputation has accrued.

2.8 Summary and Research Gap

Table 2.1 summarizes the reviewed approaches according to their privacy posture, computational cost, and robustness to adaptive, stealthy free-rider strategies such as that of Zhu et al. [16].

Table 2.1: Comparative summary of existing free-rider mitigation approaches

Category	Representative Work	Privacy-Preserving	Lightweight	Robust to Advanced Attacks
Foundational & statistical detection	Fraboni et al. [3]; Lin et al. [8] (STD-DAGMM)	Yes	Moderate	Limited
Advanced attack construction	Zhu et al. [16]	N/A (attack)	N/A	N/A
Server-side audit-based	FRAD [13]; PASS [14]	Yes	Moderate	Limited–Moderate
Privacy-attack-based	FRIDA [11]	Partial ¹	No (MIA); Moderate (PIA)	Yes (esp. PIA)
Hardware/ cryptographic	TEE, PoW, zero-knowledge proofs	Yes	No	Yes
Contribution/ incentive-based	Shapley approximations; reputation systems	Yes	No	Partial

Several patterns emerge from this review. First, purely statistical and feature-based methods [3], [8] are lightweight and privacy-preserving but were explicitly shown to be circumventable by attacks constructed to match their target statistics [16], and this vulnerability propagates into the audit-based methods (FRAD, PASS) that build on similar anomaly-detection backbones. Second, the privacy-attack-based approach of FRIDA [11] achieves markedly better robustness against exactly this class of advanced,

¹FRIDA requires server-side inspection of individual client updates, in tension with secure aggregation; the authors propose a two-server architecture as a partial mitigation, and show PIA-based detection remains effective under local differential privacy.

geometry-aware attacks, but its MIA variants require client compliance in training on server-issued canary data (an assumption that a resource-constrained or adversarially adaptive free rider may violate or exploit), and both MIA and PIA variants require the server to inspect individual, non-aggregated client updates—placing FRIDA in direct tension with secure aggregation, one of FL’s core privacy-preserving mechanisms. Third, hardware- and cryptography-based methods offer strong guarantees largely independent of the free rider’s statistical sophistication, but their cost, hardware dependency, and poor fit for heterogeneous, resource-constrained deployments make them impractical for many real-world FL scenarios. Finally, contribution- and incentive-based methods provide a longitudinal, economically grounded perspective but are either computationally intractable in exact form or vulnerable to strategic, time-varying adversarial behavior.

Across all categories, no reviewed method simultaneously satisfies three properties that are jointly necessary for practical deployment in large-scale, real-world FL systems: (i) *privacy preservation*, in the sense of not requiring the server to inspect raw, individually attributable client updates or client data, and remaining compatible with mechanisms such as secure aggregation; (ii) *lightweight operation*, avoiding the training of auxiliary models (autoencoders, shadow models) or the imposition of additional client-side training and communication burdens that scale poorly with model size or client population; and (iii) *robustness to dynamic, adaptive adversaries*, remaining effective against free-rider strategies explicitly designed—as in Zhu et al. [16]—to evade previously published defenses, and against client populations and data distributions that shift over the course of training. Methods that are lightweight and privacy-preserving (feature-based statistics) sacrifice robustness to advanced attacks; methods that are robust and reasonably privacy-conscious (FRIDA) sacrifice lightweight operation and full compatibility with secure aggregation; and methods that are robust and lightweight in the security sense (hardware/cryptographic) sacrifice deployability, cost-effectiveness, and applicability to heterogeneous edge and IoT clients.

This unresolved three-way trade-off constitutes the central research gap motivating this thesis. The remainder of this work develops a free-rider detection and mitigation framework explicitly designed to occupy the previously unaddressed region of this design space: relying only on information that is either already available to the server or obtainable without violating standard FL privacy guarantees, avoiding the training of auxiliary detection models or reliance on client-side compliance with server-issued probing data, and evaluated explicitly against adaptive, geometry-aware free-rider strategies across dynamic, non-IID, and large-scale client populations.

Chapter 3

Proposed Methodology

3.1 Design Objective and Information Boundary

Suspicion-Weighted Trap Detection with Cumulative Penalties (SWT-CP) is a server-orchestrated challenge–response mechanism for free-rider mitigation. The server occasionally sends a selected client a randomized model in place of the current global model. This secret model is called a *trap*. An honest client applies the prescribed local optimizer to whichever model it receives, whereas a free rider fabricates a response from its locally visible model history. Because the client is never told whether a received model is a trap, the intervention changes the attacker’s input without changing the client API.

The detector trusts neither local accuracy nor loss, reported training time, hardware counters, number of steps, nor any other client-supplied claim. Its decision inputs are derived by the server from the returned model: whether the update is effectively zero, the norm of the flattened update, and the relative distribution of update magnitude across parameter tensors. Ground-truth free-rider labels and the configured attack name are used only to evaluate the simulator; they are not available to the detection rule. Cosine similarity and local metrics are logged for analysis but do not affect flags, penalties, state transitions, anchor selection, or removal.

This boundary makes the method lighter than an auxiliary autoencoder or a client-side proof-of-work mechanism. It does not, however, constitute a cryptographic proof of training. It also assumes that the server can inspect an individual returned model during an audit; ordinary secure aggregation would require a separate auditable channel or a privacy-preserving computation of the required statistics.

3.2 System and Threat Model

Let N be the registered clients and T the communication rounds. Client i receives $w_{t,i}^{\text{send}}$ in round t and returns $w_{t,i}^{\text{ret}}$. The server derives

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \quad r_{t,i} = \|\Delta_{t,i}\|_2. \quad (3.1)$$

In an ordinary round $w_{t,i}^{\text{send}} = w_t^g$; a trapped client instead receives w^{trap} . Trap-derived updates are never aggregated because they do not share the global starting point.

The implementation evaluates four attack behaviors through the same server rule:

- **FR1—replay:** return the last model observed by that client; on first participation, return the current received model.
- **FR2—bounded random update:** add independent uniform noise in a fixed small range to each floating-point parameter of the received model.
- **FR3—historical average:** average the latest five models received by that client, including any unlabelled traps.
- **FR4—trajectory prediction:** estimate an expected update from the latest five differences between consecutively received models and add Gaussian noise equal to 10% of the estimated norm. Before history exists, use a small random cold-start update.

The attacker sees its identity, round number, received model, public configuration, its own received-model history, and local randomness. It receives no trap indicator. Collusion, defense-aware adaptation, on-off participation, and partially honest local training are outside the evaluated scope.

3.3 Robust Two-Signal Detector

3.3.1 Two-sided magnitude evidence

For a reference set of update norms R , SWT-CP computes

$$m_r = \text{median}(R), \quad (3.2)$$

$$d_r = \text{median}_{r \in R} |r - m_r|, \quad (3.3)$$

$$z_{t,i}^{\text{norm}} = \frac{r_{t,i} - m_r}{\max(d_r, 0.05)}. \quad (3.4)$$

The magnitude signal fires when $|z_{t,i}^{\text{norm}}| > 3$. The absolute value is essential: replay may create an excessively large response, whereas an adaptive fabricated update may be abnormally small. The MAD floor prevents a nearly identical reference population from turning negligible numerical differences into extreme standardized scores.

3.3.2 Scale-free layer profile

For each floating parameter tensor k , the server forms

$$p_{t,i,k} = \frac{\|\Delta_{t,i,k}\|_2}{\sum_j \|\Delta_{t,i,j}\|_2}. \quad (3.5)$$

The vector $p_{t,i}$ records how update energy is distributed across layers independently of total scale. A coordinate-wise median reference profile is constructed from the applicable comparison population. The Euclidean distance from this center is standardized by median and MAD, with a floor of 0.01; the profile signal fires above 3.

A single magnitude or profile anomaly is sufficient to quarantine an unflagged client for confirmation, but it contributes no magnitude penalty. Only a response that fails both tests on the same probe creates *joint gradient evidence*. This conjunction was introduced because persistent honest clients can occupy the tail of either one-dimensional statistic.

3.3.3 Exact send-back evidence

If $r_{t,i} < 10^{-6}$, the response is treated as an exact or near-exact send-back. It immediately adds five permanent points and sends the client directly to the suspicious state. This check operates in every phase, including warm-up. It is evaluated separately from nonzero magnitude/profile evidence and is never erased by rehabilitation.

3.4 Client States and Evidence Accounting

Each active client occupies one of three operational states:

Unflagged (U) no unresolved anomaly; the update is normally eligible for aggregation unless trapped in the current round.

Candidate (C) at least one nonzero anomaly has occurred and the client awaits fresh-trap confirmation. Candidate updates are quarantined.

Suspicious (S) repeated joint evidence or an exact send-back has confirmed elevated suspicion. The client is probed frequently.

The fixed removal threshold is 15 points. A joint anomaly contributes two points in initial coverage, one point in later surveillance, and three points in candidate/suspicion probing. A magnitude-only or profile-only anomaly contributes zero points. Nonzero evidence cannot remove a client until at least ten relevant episode probes have been observed. Three exact send-backs reach 15 points independently of that gate.

For C and S clients, SWT-CP maintains episode probe and joint-failure counts. After at least ten probes, a client is rehabilitated when its joint-failure rate is at most 10%. Rehabilitation returns it to U , clears the current magnitude episode and resets episode counters, while retaining exact-send-back points. If rehabilitation and nonzero removal become eligible simultaneously, rehabilitation is applied first.

Table 3.1: Principal SWT-CP v8 parameters

Parameter	Meaning	Value
n_w	warm-up rounds	10
ρ	nominal trap group fraction	0.10
k_r, k_p	norm/profile thresholds	3.0, 3.0
ϵ_r, ϵ_p	norm/profile MAD floors	0.05, 0.01
δ_0	exact send-back penalty	5
δ_{cov}	initial joint penalty	2
δ_Y	surveillance joint penalty	1
δ_S	suspicion joint penalty	3
P_{max}	removal threshold	15
q_{min}	rehabilitation probe minimum	10
η_{max}	rehabilitation failure rate	0.10

3.5 Probing Schedule

3.5.1 Warm-up

Rounds 1–10 use standard FedAvg. The exact send-back test remains active, but nonzero magnitude/profile scoring is disabled so early optimization transients do not create candidates.

3.5.2 Initial double coverage sweep

After warm-up, clients are shuffled into groups of approximately 10%. The server generates two distinct traps, A and B, from a global base. Each trap is frozen for one

complete pass. Two groups are probed per round: a randomized first pass gives every client trap A once, and an independently randomized second pass gives every client trap B once, avoiding repeated group pairs where possible. Thus every client receives two secret probes during ten coverage rounds.

Responses are buffered until a full pass is complete. Statistics are computed separately for A and B, ensuring that responses to different traps do not define one another's baseline. Exact send-backs add five points each; each joint norm/profile failure adds two; a one-signal anomaly creates candidate evidence but no magnitude points. Ambiguous clients enter C , while sufficiently repeated evidence enters S . The initial double sweep is performed once.

3.5.3 Post-coverage surveillance

All currently unflagged clients are shuffled into exactly ten Y groups. One group is audited per round using the cycle's frozen surveillance trap and excluded from that round's aggregation. After all groups are visited, a new shuffled cycle begins. Any anomaly sends a client to C and permanently disqualifies it from anchor service; only a joint anomaly adds one point.

3.5.4 Candidate and suspicion probing

When S is nonempty, the number of suspicion groups is

$$G_S = \max\left(1, \left\lfloor \frac{|S|}{10} \right\rfloor\right). \quad (3.6)$$

Ten is a target, not a maximum group size. Candidates are distributed across the same groups, or one confirmation group is formed when only candidates exist. The implementation reshuffles suspicious and candidate membership across groups each round. Each group receives a distinct fresh trap.

Each group receives up to three active anchors, subject to a system-wide maximum of ten. Suspect and candidate responses are compared only with anchors that received the same trap. Anchor-relative medians and MADs define both decisions. A joint failure adds three points; a normal candidate response clears C immediately. Two joint failures confirm a candidate as suspicious by default.

3.6 Rotating Anchor Roster

The v8 change replaces fixed panels with a rotation queue. Eligible anchors are active, unflagged clients whose lifetime anomaly count is zero. An anchor is not reused until the eligible queue has been traversed where population size permits. Clients in the current Y group cannot simultaneously anchor. Off-duty roster members receive the ordinary global model and contribute normally; only active anchors receive a trap and are excluded.

Rotation reduces repeated comparison against one unlucky panel, and Y auditing can remove an anomalous client from future eligibility. Nevertheless, eligibility is absence of evidence, not proof of honesty. Chapter 4 shows that adaptive attackers can evade surveillance, enter the roster, and contaminate panels.

3.7 Aggregation, Removal, and Checkpointing

Every trap recipient—suspect, candidate, anchor, or Y client—is excluded from aggregation in that round. Candidates remain quarantined through confirmation. Permanent removal occurs when exact-send-back evidence reaches 15, or when total retained points reach 15 after the ten-probe nonzero-evidence gate and rehabilitation does not apply.

The simulator saves a checkpoint after every completed round, retains the newest five while a run is incomplete, and deletes checkpoints after successful completion. Checkpoints include model and attacker-visible history, client states, penalties, episode counters, buffered coverage evidence, and anchor rotation. Logs record model utility, round time, memory, trap assignments, state transitions, removals, rehabilitation, and per-client detection statistics.

3.8 Computational Characteristics and Limitations

For d model parameters, deriving the norm and profile costs $O(d)$ per inspected update. Median/MAD construction over m observations costs $O(m \log m)$ by sorting and $O(m)$ temporary scalar storage. Persistent state is $O(N)$. No detector model is trained and the client performs only its ordinary optimizer on the model received. The main utility cost is lost aggregation capacity from trap recipients and candidates.

SWT-CP observes statistical consequences rather than proving training. Median/MAD has a theoretical 50% scalar breakdown point, but practical separation can fail earlier

when honest non-IID responses are broad or multimodal. Anchor rotation cannot help if undetected attackers qualify as anchors. Per-client inspection conflicts with ordinary secure aggregation. Evaluation is limited to MNIST, two seeds, two attacker shares, full participation, and always-on non-colluding attacks; CIFAR-10, dropout, collusion, on-off behavior, selfish partial training, and defense-aware adaptation remain future work.

3.9 Summary

SWT-CP v8 combines secret two-trap coverage, two-sided norm and layer-profile evidence, candidate confirmation, frequent suspicion probing, rotating anchors, continued surveillance, cumulative penalties, and rehabilitation. Its rules are attack-independent and require no trusted client-reported metric. The next chapter evaluates both its strong IID performance and the limits revealed by heterogeneous data and adaptive FR4 behavior.

Chapter 4

Results and Discussion

4.1 Experimental Questions

The evaluation addresses four questions. First, can SWT-CP detect replay, random-update, historical-average, and adaptive free riders without knowing which attack is present? Second, how sensitive is detection to the random seed and attacker proportion? Third, does Dirichlet non-IID partitioning increase false positives or create false negatives? Fourth, what model-utility and execution costs accompany the probing schedule?

4.2 Experimental Setup

All reported v8 experiments use MNIST, 100 clients, full participation, 100 communication rounds, three local epochs, batch size 32, SGD with learning rate 0.01 and momentum 0.9, and a convolutional classifier. IID partitions distribute examples uniformly. Non-IID partitions use a Dirichlet label allocation with concentration $\alpha = 0.5$. The matrix contains attacker shares of 30% and 40%, seeds 42 and 43, and FR1–FR4, yielding 32 completed runs. No result from v4–v7 is pooled with v8.

Precision, recall, and F1 treat permanent removal as the positive decision. Thus TP is a removed free rider, FP a removed honest client, and FN an active free rider at round 100. Final candidates are not counted as false removals. This distinction is important because candidates created in round 100 have no later confirmation round. Global accuracy is evaluated on the MNIST test set. Resource measurements are process-level measurements from the simulator; resumed-run wall-clock totals may describe only the resumed segment, so average round time is used for cross-run comparison.

Table 4.1: Aggregate v8 results across FR1–FR4 for each setting

Partition	Attackers/seed	TP	FP	FN	Precision	Recall	F1	Mean accuracy
IID	30%/42	120	8	0	93.75%	100%	96.77%	98.95%
IID	30%/43	120	1	0	99.17%	100%	99.59%	98.92%
IID	40%/42	160	6	0	96.39%	100%	98.16%	98.86%
IID	40%/43	160	0	0	100%	100%	100%	98.89%
Non-IID	30%/42	120	10	0	92.31%	100%	96.00%	98.88%
Non-IID	30%/43	120	9	0	93.02%	100%	96.39%	98.88%
Non-IID	40%/42	120	7	40	94.49%	75%	83.62%	98.84%
Non-IID	40%/43	120	8	40	93.75%	75%	83.33%	98.88%

4.3 IID Results

All 560 free riders in the four IID configurations were removed. Seed 43 was particularly strong: one honest client was removed across the 30% experiments and none across the 40% experiments. Seed 42 produced eight and six honest removals respectively. The gap shows that a single seed would give a misleading estimate of precision even under IID partitioning. Across all IID runs, pooled removal precision was $560/(560 + 15) = 97.39\%$, recall was 100%, and F1 was 98.68%.

FR1–FR4 all remained detectable under IID data. Final accuracy varied by only 0.14 percentage points across the aggregate settings. The defense therefore preserved utility in these experiments despite excluding trapped and candidate updates. However, the seed-42 false removals demonstrate that joint evidence and rotation reduce rather than eliminate honest tails.

4.4 Non-IID Results

Table 4.2 presents each non-IID attack as “free riders removed / honest clients removed.” FR1–FR3 achieved complete recall in every setting. FR3 was the least precise, producing three or four false removals per run. FR4 exhibited a qualitatively different transition: it was fully detected at 30% but completely missed at 40% under both seeds.

Table 4.2: Detailed non-IID results (free riders removed / honest clients removed)

Attack	30%/seed 42	30%/seed 43	40%/seed 42	40%/seed 43
FR1	30/3	30/1	40/1	40/3
FR2	30/2	30/3	40/1	40/2
FR3	30/3	30/3	40/4	40/3
FR4	30/2	30/2	0/1	0/0

At 30%, all FR4 clients entered the candidate/suspicion pipeline and were removed around rounds 30–32. Joint evidence appeared in 76.01% of trapped FR4 observations for seed 42 and 81.28% for seed 43. At 40%, the corresponding rates fell to 0% and 2.86%; no attacker accumulated enough repeated evidence. Seed 42 never flagged any of its 40 FR4 clients. Seed 43 flagged 18 at least once, but none received adequate confirmation. Every one of the 80 attackers across the two 40% FR4 runs remained active at round 100.

This discontinuity is visible during initial coverage. Median FR4 norm z-scores were -2.65 and -3.10 at 30%, but only -1.55 and -1.81 at 40%. Increasing contamination changed the median/MAD and layer-profile references enough to make the malicious cluster appear less exceptional, even though its raw update norms remained much smaller. In the seed-42 40% run, the median trapped norm was 0.0365 for FR4 and 0.4824 for honest clients, a factor of approximately 13. The detector failed not because responses were physically similar, but because the reference and joint-decision mechanism did not convert that separation into evidence under the broad non-IID mixture.

4.5 Anchor Contamination Case Study

The 40% FR4 failure also invalidated the operational interpretation of an eligible anchor. Because no FR4 client had prior evidence, attackers entered the never-flagged roster. In the seed-42 run, honest client 85 became suspicious and was compared with 33 anchor assignments over its episode; 10 were free riders. At rounds 66, 68, and 73, two of its three anchors were FR4 clients. Those exact three malicious-majority panels produced joint norm/profile failures and added three points each. Combined with six earlier points, client 85 reached 15 and was removed at round 73.

This trace establishes a causal failure rather than a general suspicion: rotating an unverified roster distributes contamination but does not remove it. Three anchors tolerate one bad reference, not two. A client that has never been detected is not necessarily a positively verified honest client.

4.6 False Positives and Seed-Specific Clients

Non-IID false removals were concentrated rather than uniformly random. Under seed 42, client 65 was removed in six non-IID attack/configuration combinations and client 74 in five. Under seed 43, client 81 was removed seven times and client 29 five times.

Repetition across FR1–FR4 indicates partition-specific honest update geometry rather than an attack-specific response. The detector repeatedly interprets certain legitimate label distributions as joint norm/profile anomalies.

At round 100 the non-IID settings contained between 17 and 34 candidates and one to four suspects when aggregated across the four attacks. Most final candidates entered in the last round and are boundary artifacts rather than confirmed removals. Final suspects are more important: they show that removal precision alone understates temporary quarantine and future false-removal risk.

4.7 Model Utility

Mean final accuracy remained between 98.84% and 98.95% in every aggregate setting. The individual-run range was 98.82–98.95%. Notably, the non-IID 40% FR4 runs still achieved 98.84% and 98.88% despite detecting no attackers. FR4 returns low-magnitude fabricated updates; with 60 honest clients remaining, these responses mainly dilute learning rather than immediately destroy classification accuracy. Consequently, high test accuracy is not evidence of successful free-rider detection.

4.8 Runtime and Resource Use

Table 4.3: Execution profile aggregated across four attacks

Partition	Attackers/seed	Mean s/round	Approx. batch hours	Peak RAM (MB)
IID	30%/42	50.8	5.65	1074
IID	30%/43	43.4	4.82	1150
IID	40%/42	59.6	6.62	3172
IID	40%/43	54.3	6.04	3237
Non-IID	30%/42	46.9	5.21	887
Non-IID	30%/43	46.0	5.11	891
Non-IID	40%/42	42.4	4.72	1079
Non-IID	40%/43	44.8	4.98	1022

The IID 40% runs recorded CUDA memory, while the other collected batches executed on CPU. GPU execution did not yield proportional speedup because clients are simulated sequentially and much of the workload and orchestration remains serial. FR4 40% non-IID was relatively costly within its batch because all attackers remained active: the seed-42 run averaged 89 aggregated clients per round and ended with 99 active clients, compared with roughly 55–56 aggregated and 56–59 active clients after

successful FR1–FR3 removal. Peak recorded system-memory use approached 95% in some CPU runs, so host memory was a practical execution constraint.

4.9 Threats to Validity

Only two seeds were evaluated, and their false-positive difference is large. The experiments use one dataset, one model family, one Dirichlet concentration, full participation, and always-on independent attackers. Reported percentages are therefore measurements of this simulation, not confidence intervals for deployment. No external defense was rerun under an identical implementation, so the results do not establish superiority over FRIDA, FRAD, PASS, or STD-DAGMM. Accuracy remains high even in a detection failure, and should not be treated as a proxy for contribution authenticity. Finally, individual-update inspection is incompatible with standard secure aggregation without additional protocol support.

4.10 Overall Interpretation

The central empirical conclusion is conditional. SWT-CP v8 is highly effective against FR1–FR3 across the tested IID and non-IID settings, and against FR4 under IID or 30% non-IID contamination. It is not robust to FR4 at 40% non-IID contamination. The repeated two-seed failure indicates a methodological boundary rather than a corrupted run. Future revisions should require positive evidence before anchor admission and exploit paired within-client responses to the two coverage traps, reducing dependence on comparisons among naturally heterogeneous clients.

Chapter 5

Conclusion

This thesis investigated whether a server can mitigate free riding using only returned model updates, without trusting client-reported accuracy, loss, training time, or resource use and without training an auxiliary detector. SWT-CP v8 addresses this objective through secret trap models, two-sided norm and layer-profile evidence, a candidate/suspicion state machine, repeated probing, rotating anchors, cumulative penalties, and rehabilitation.

The IID evaluation is encouraging: every tested free rider was removed across attacker shares of 30% and 40% and seeds 42 and 43. Seed 43 produced only one honest removal at 30% and none at 40%. FR1–FR3 also retained complete recall under non-IID partitioning. These results support the use of unpredictable server interventions and longitudinal evidence for lightweight detection of replay, random-update, and historical-average behavior.

The evaluation also revealed two limits that are as important as the successes. First, non-IID false positives concentrate on particular honest clients and recur across attacks, showing that joint statistical evidence is still sensitive to legitimate client heterogeneity. Second, FR4 exhibits a reproducible contamination transition: both non-IID 30% runs removed all attackers, whereas both 40% runs removed none. Undetected FR4 clients entered the anchor roster, and malicious-majority panels directly contributed to an honest removal. Rotation is therefore not equivalent to trust establishment.

The immediate research priority is to replace negative anchor eligibility (“never flagged”) with positive, repeated validation, and to use differential responses from the same client across independent traps so that each client partly serves as its own non-IID control. A third seed and intermediate 35% FR4 experiment should confirm the tran-

sition. Broader validation should include CIFAR-10, additional Dirichlet concentrations, client dropout, on-off and partial-training free riders, collusion, defense-aware adaptation, and comparisons with implemented baselines. A secure protocol is also needed if per-client auditing is to coexist with secure aggregation.

SWT-CP should thus be understood as a computationally simple and empirically strong detector within a defined operating region, not as proof that a client trained. Its main contribution is both constructive and diagnostic: it demonstrates where secret statistical traps work, and identifies the precise combination of heterogeneity, adaptive fabrication, and contaminated references under which they fail.

References

- [1] K. Bonawitz et al., “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. DOI: 10.1145/3133956.3133982
- [2] K. Bonawitz et al., “Towards federated learning at scale: System design,” in *Proceedings of Machine Learning and Systems*, vol. 1, 2019, pp. 374–388. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/bd686fd640be98efaae0091fa301e613-Abstract.html
- [3] Y. Fraboni, R. Vidal, and M. Lorenzi, “Free-rider attacks on model aggregation in federated learning,” in *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, A. Banerjee and K. Fukumizu, Eds., ser. Proceedings of Machine Learning Research, vol. 130, PMLR, Apr. 2021, pp. 1846–1854. [Online]. Available: <https://proceedings.mlr.press/v130/fraboni21a.html>
- [4] A. Ghorbani and J. Zou, “Data shapley: Equitable valuation of data for machine learning,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 2242–2251. [Online]. Available: <https://proceedings.mlr.press/v97/ghorbani19c.html>
- [5] A. Hard et al., “Federated learning for mobile keyboard prediction,” *arXiv preprint arXiv:1811.03604*, 2018. [Online]. Available: <https://arxiv.org/abs/1811.03604>
- [6] R. Jia et al., “Efficient task-specific data valuation for nearest neighbor algorithms,” *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1610–1623, 2019. DOI: 10.14778/3342263.3342637
- [7] P. Kairouz et al., “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. DOI: 10.1561/22000000083

- [8] J. Lin, M. Du, and J. Liu, “Free-riders in federated learning: Attacks and defenses,” *arXiv preprint arXiv:1911.12560*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.12560>
- [9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, PMLR, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [10] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, “Exploiting unintended feature leakage in collaborative learning,” in *2019 IEEE Symposium on Security and Privacy*, 2019, pp. 691–706. DOI: 10.1109/SP.2019.00029
- [11] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, “FRIDA: Free-rider detection using privacy attacks,” *arXiv preprint arXiv:2410.05020*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.05020>
- [12] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18. DOI: 10.1109/SP.2017.41
- [13] B. Wang, H. Li, X. Liu, and Y. Guo, “Frad: Free-rider attacks detection mechanism for federated learning in aiot,” *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. DOI: 10.1109/JIOT.2023.3298606
- [14] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, “PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack,” *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2023. DOI: 10.1109/JIOT.2023.3298557
- [15] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated learning with non-iid data,” *arXiv preprint arXiv:1806.00582*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.00582>
- [16] Z. Zhu, J. Shu, X. Zou, and X. Jia, “Advanced free-rider attacks in federated learning,” in *Proceedings of the 1st NeurIPS Workshop on New Frontiers in Federated Learning: Privacy, Fairness, Robustness, Personalization and Data Ownership*, 2021, p. 10.